M&A OPERATING SYSTEM

Product Design

MAOS Internet Access MCP Server

Draft v1.0 · June 2026

Andrew Bush (www.maoperatingsystem.com/bio-andrew-bush)

Contents

About This Document

This document is part of the **M&A Operating System** product design specification series. It describes the intended design, architecture, and behavior of the platform within. It is intended for internal distribution across product, engineering, and design review functions.

This is a living specification. It reflects design intent at the time of publication and will be updated as the product evolves.

About M&A Operating System

M&A Operating System is a boutique advisory firm focused on helping organizations modernize the operational foundations required for scalable analytics, trusted enterprise information, and practical AI adoption.

Our work sits at the intersection of enterprise data strategy, information architecture, operating model modernization, and transformation execution. We specialize in helping organizations design practical, business-aligned approaches to enterprise information management that support operational scalability, analytics enablement, governance, and modern AI capabilities.

A core part of our approach is the recognition that successful data and AI initiatives are not driven solely by technology platforms. Long-term success depends on how well organizations structure, model, govern, operationalize, and align information with real business processes and decision-making.

Our Product Design document series reflects this philosophy. These documents are intended to provide practical frameworks, methodologies, architectural concepts, and implementation guidance that help organizations move beyond theoretical strategy into executable operational design.

M&A Operating System's approach combines: - Enterprise data and AI strategy - Subject-based data modeling and information architecture - Semantic and operational data design - Governance and organizational alignment - Operating model modernization - Transformation execution and delivery leadership - Practical AI operationalization and readiness

Our goal is simple: help organizations build practical, scalable, and trusted information foundations that improve operational effectiveness, accelerate analytics and AI adoption, and support better business outcomes.

Find Out More

To learn more about M&A Operating System, explore the platform, or speak with our team: maoperatingsystem.com

01 — Overview

Product: MAOS Internet Access MCP Server
Version: 1.0
Date: 2026-06-16
Author: Andrew Bush / M&A Operating System

Problem Statement

AI agents and conversational AI interfaces have no controlled, self-hosted mechanism for accessing live internet content during interactions. Without it, they operate only on training data and users must manually retrieve and paste web content into conversations. Autonomous agents have no mechanism to retrieve current information during pipeline execution.

Solution

A self-hosted MCP server that exposes web search and page fetch as production-grade, infrastructure-controlled capabilities. Built-in entitlements and site classification govern what internet content AI agents and users can access and consume. No external API dependency is required for core operation.

Primary Consumers

AI Chat Platform — the primary conversational AI front end; registers this server in its MCP tool registry as **MCP Internet Fetch & Search** to provide end users with real-time web search and page fetch during conversations.

Autonomous AI agents — software agents executing within agentic pipelines or multi-agent orchestrations that require live internet content as a step in task execution. The agent issues tool calls and receives structured content without human involvement.

Deployment Model

The server is deployed as shared enterprise infrastructure by a platform or AI infrastructure team. It is not a commercial SaaS product. It runs self-hosted within the enterprise environment and is consumed by any MCP-compatible client — AI assistants, coding agents, orchestration frameworks — operating within that environment.

Layered Control Model

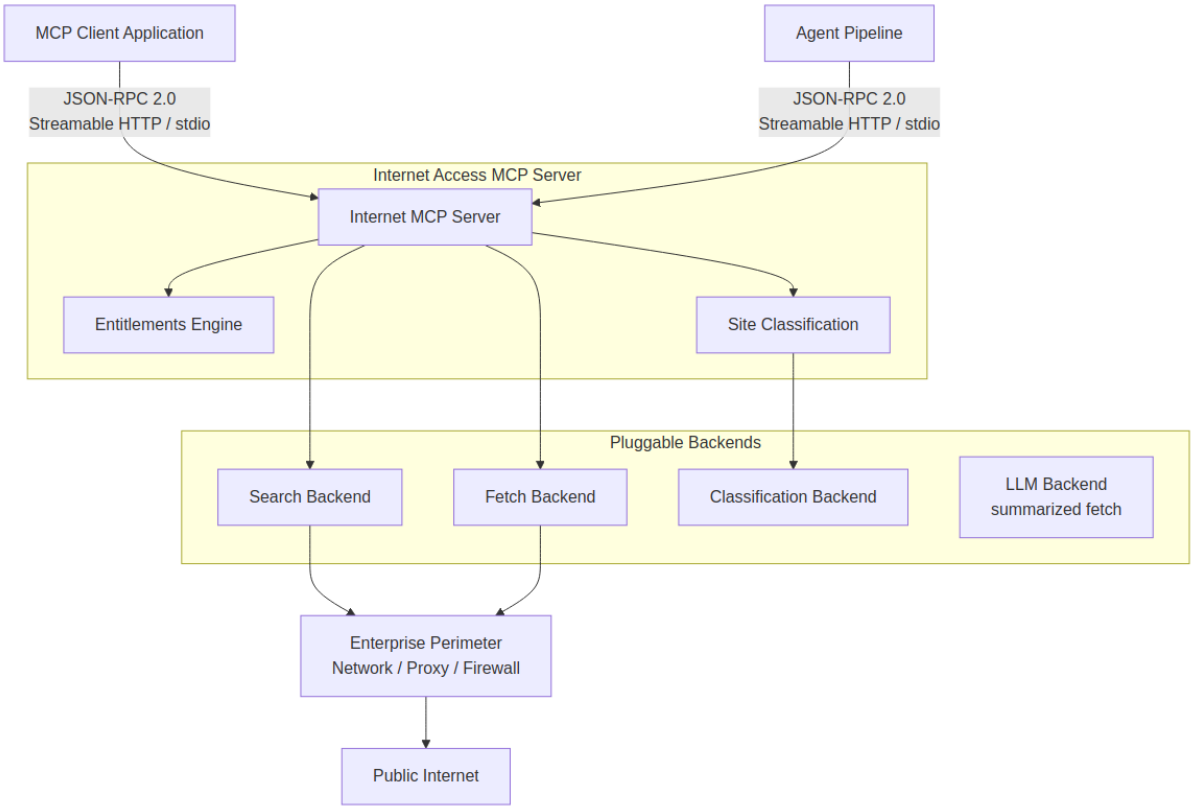
The server operates within a two-layer control model.

Layer 1 — Enterprise perimeter controls. Network, proxy, firewall, and identity policies that govern all outbound internet access for the infrastructure on which the server runs. The server inherits and is subject to these controls automatically by virtue of where it is deployed.

Layer 2 — Server-managed controls. Entitlements and site classification operating specifically against AI interactions — governing what content the server will search, fetch, and return to agents and interfaces. These controls are additive to the enterprise perimeter, not a replacement for it.

This layering means the server can be deployed within an organization that already enforces outbound internet policy, and the server's own controls then address the AI-specific access surface on top of that foundation.

System Context



Success Metric

Volume of web pages successfully fetched per deployment — demonstrating active grounding of AI interactions in live internet content.

Current State Without This Product

No controlled mechanism exists. Users manually copy and paste web content into conversations. Autonomous agents cannot retrieve live internet content during execution and either fail or produce responses based solely on training data.

Decisions Log

ID	Decision
D-001	The server is a general-purpose MCP infrastructure component, not tied to any specific platform or application stack
D-002	Search and fetch are the MVP; entitlements and site classification are included in MVP as safety controls, not Phase 2 features
D-003	Site classification is a safety gate — content that fails classification is blocked server-side before being returned to the LLM
D-004	Classification metadata is returned alongside permitted content in all responses, even when content passes the safety threshold
D-005	Four fetch output formats in target state: raw, markdown, chunked, summarized
D-006	Server-managed controls are explicitly additive to enterprise perimeter controls, not a replacement
D-007	Authenticated fetch targets enterprise IdP (v1) and consumer OAuth providers (v2); not included in MVP
D-008	OAuth 2.0 OBO flow for enterprise IdP; OAuth 2.1 with PKCE for consumer providers; no token passthrough permitted
D-009	Search backend is pluggable behind a standard internal interface
D-010	Classification backend is pluggable; most-restrictive-wins when multiple providers are configured
D-011	All successful search and fetch responses include a provenance block (URL/query, timestamp, size, content hash, backend) returned to the caller and independently recorded in the audit log — establishing a verifiable chain of custody from request to response

02 — Core Capabilities

Product: MAOS Internet Access MCP Server
Version: 1.0
Date: 2026-06-16
Author: Andrew Bush / M&A Operating System

Capability Model

The server exposes five capability areas as MCP tools. Each is described below with its purpose, inputs, outputs, behavior, and acceptance criteria.

1. Search

Purpose

Accept a natural language or keyword query and return a ranked list of web search results drawn from the configured search backend.

Inputs

Parameter	Type	Required	Description
query	string	Yes	The search query
num_results	integer	No	Number of results to return; default and maximum configurable at deployment
language	string	No	Language preference for results
time_range	string	No	Restrict results by recency (day, week, month, year, any)

Outputs

A ranked list of results, each containing: - title - url - snippet - source domain - site classification metadata (category, reputation score, safety flag) — see Capability 4

Each response also includes a **provenance block** recording: the query string, execution timestamp (UTC), backend identifier, result count before and after filtering, and a SHA-256 hash of the filtered result set. The provenance block is returned to the caller and independently written to the audit log.

Behavior

The search backend is configurable at deployment time. The server abstracts the backend — callers issue a standard MCP tool call regardless of which backend is active. Results are filtered through the active entitlement policy before being returned; results for blocked or restricted domains are silently excluded from the response. Results whose site classification falls below configured safety thresholds are excluded or flagged according to deployment policy.

Unhappy Paths

- Backend unavailable: returns a structured error with reason; does not return partial or cached results unless caching is explicitly configured
- Query returns zero results after entitlement and classification filtering: returns empty results list with a filter-applied indicator
- Query violates entitlement policy entirely: returns a policy rejection response with no results

Acceptance Criteria

- Given a valid query, when the tool is called, then a ranked list of results is returned within the configured timeout
- Given a valid query, when results include domains on the deny list, then those results are excluded before the response is returned
- Given a valid query, when results include URLs with a safety classification below threshold, then those results are excluded or flagged per deployment policy
- Given the search backend is unavailable, when the tool is called, then a structured error is returned and no partial results are served
- Given a query returning zero results after filtering, when the tool is called, then an empty results list with a filter-applied indicator is returned

---

2. Fetch

Purpose

Accept a URL and return the page content in one of four caller-selected output formats, suitable for consumption by an LLM or downstream processing step.

Inputs

Parameter	Type	Required	Description
url	string	Yes	The URL to fetch
format	enum	No	Output format: raw, markdown, chunked, summarized. Default: markdown

Parameter	Type	Required	Description
chunk_size	integer	No	Token or character limit per chunk when format is chunked
start_chunk	integer	No	Chunk index to begin from, enabling pagination through large pages

Output Formats

Raw — The unprocessed HTTP response body as returned by the target server. Preserves all HTML, scripts, and structure. Intended for callers that perform their own parsing.

Markdown — The page content converted to clean Markdown. Navigation, advertisements, headers, footers, and boilerplate are stripped. The result is LLM-ready structured text preserving headings, lists, links, and code blocks.

Chunked — The Markdown-converted content segmented into discrete chunks of a configurable size. Each chunk includes its index and the total chunk count. Enables agents to paginate through large pages without exceeding context window limits.

Summarized — An LLM-generated summary of the page content. The server passes the Markdown-converted content to a configured language model and returns the summary. The model used is configurable at deployment time.

All fetch formats return a **provenance block** alongside the content: the final URL (after redirects), fetch timestamp (UTC), raw content size in bytes, SHA-256 hash of the raw response body before format conversion, HTTP status code, and the original URL if a redirect occurred. The provenance block is returned to the caller and independently written to the audit log.

Behavior

Before executing a fetch, the server checks the URL against the active entitlement policy. If the URL or its domain is on the deny list, the fetch is rejected and a policy response is returned. If the URL's site classification falls below the configured safety threshold, the fetch is rejected or the response is flagged according to deployment policy. Fetch is performed using the server's own HTTP client — no browser automation is required for static content. JavaScript-rendered pages require a configured browser automation backend.

Unhappy Paths

- URL blocked by entitlement policy: returns a policy rejection response with the reason
- URL blocked by site classification: returns a classification rejection response
- Target server returns non-200 response: returns a structured error including the HTTP status code
- Target server times out: returns a timeout error with the configured timeout value
- Summarized format requested but no LLM backend configured: returns an error indicating the capability is not available in this deployment

- Chunked format requested with start_chunk beyond available chunks: returns an error indicating the chunk index is out of range

Acceptance Criteria

- Given a valid permitted URL, when fetch is called with format markdown, then clean Markdown content is returned with boilerplate stripped
- Given a valid permitted URL, when fetch is called with format raw, then the unprocessed HTTP response body is returned
- Given a valid permitted URL, when fetch is called with format chunked, then content is returned in segments with index and total chunk count in each response
- Given a valid permitted URL, when fetch is called with format summarized and an LLM backend is configured, then a summary of the page content is returned
- Given a URL on the deny list, when fetch is called, then a policy rejection response is returned and no content is fetched
- Given a URL with a site classification safety flag below threshold, when fetch is called, then the fetch is rejected per deployment policy
- Given a target server timeout, when fetch is called, then a structured timeout error is returned
- Given a chunked fetch with start_chunk beyond available chunks, when fetch is called, then an out-of-range error is returned

---

3. Entitlements

Purpose

Define and enforce domain and URL-level access controls that govern what internet content the server will search and fetch on behalf of AI agents and users. These controls are specific to AI interactions and are additive to any enterprise perimeter controls already in place.

Configuration Model

Entitlement rules are defined by the Platform Administrator at deployment time through server configuration. Rules are not modifiable at runtime by agents or users.

Rule Type	Description
Domain allow list	Only domains on this list are accessible. All others are blocked.
Domain deny list	Domains on this list are always blocked. Other domains are accessible subject to classification.

Rule Type	Description
URL pattern rules	Allow or deny rules applied at the URL path level using pattern matching
Default posture	When no allow list is defined: open (block only deny-listed domains) or closed (block all except allow-listed domains)

Behavior

Entitlement checks run before any search result is returned or any fetch is executed. Blocked content is never retrieved or returned. The caller receives a structured policy response indicating the request was blocked; the reason (domain blocked, URL pattern blocked) is included in the response.

Search results for blocked domains are excluded from the result list before the response is returned — the caller does not see them and is not informed of their existence unless inspection-mode logging is enabled for audit purposes.

Acceptance Criteria

- Given a domain deny list is configured, when a search or fetch is requested for a blocked domain, then the request is rejected with a policy response
- Given a domain allow list is configured, when a search or fetch is requested for a domain not on the list, then the request is rejected with a policy response
- Given URL pattern rules are configured, when a fetch is requested for a URL matching a deny pattern, then the request is rejected
- Given an open default posture with no allow list, when a fetch is requested for a domain not on the deny list, then the request proceeds subject to classification checks
- Given a closed default posture, when a fetch is requested for a domain not on the allow list, then the request is rejected

---

4. Site Classification

Purpose

Provide category and reputation metadata for URLs accessed through search and fetch, enabling the server to enforce content safety policies and prevent LLMs from consuming dangerous, malicious, or policy-violating content.

Classification Dimensions

Dimension	Description
Content category	Topical classification of the site (e.g. news, finance, technology, adult, gambling) aligned to a standard taxonomy
Reputation score	A numerical score indicating the trustworthiness and safety history of the domain
Safety flags	Explicit flags for malware, phishing, spam, hate speech, and other threat categories
Classification source	The provider or database that supplied the classification

Behavior

Classification is checked at two points: when search results are assembled, and before a fetch is executed. The deployment configuration defines threshold policies — which categories are blocked, which reputation score ranges are permitted, and which safety flags result in an automatic block.

Classification metadata is returned alongside search results and fetch responses, enabling consuming agents and UIs to make their own downstream policy decisions in addition to the server's enforcement. This supports scenarios where the enterprise wants both server-side safety enforcement and agent-layer awareness of content classification.

Classification data is sourced from one or more configured classification providers. The classification backend is pluggable at deployment time. Where multiple providers are configured, the most restrictive classification applies.

Acceptance Criteria

- Given a search result URL with a safety flag for malware, when results are assembled, then that result is excluded per deployment policy
- Given a fetch request for a URL classified in a blocked category, when the fetch is requested, then a classification rejection response is returned
- Given a search result URL with a reputation score below the configured threshold, when results are assembled, then that result is excluded or flagged per deployment policy
- Given a permitted URL, when fetch is called, then classification metadata is returned alongside the content in the response
- Given multiple classification providers are configured, when classifications conflict, then the most restrictive classification applies

5. Authenticated Fetch

Purpose

Enable the server to retrieve content from websites that require an authenticated user session — such as enterprise intranet pages, subscription services, or any site protected by OAuth or SSO — on behalf of a user who has legitimate access to that content.

Authentication Modes

Enterprise IdP (OAuth 2.1 / SAML / OIDC) The server accepts an access token issued by the enterprise identity provider (Entra ID, Okta, or any OIDC-compliant IdP) passed in the tool call context. For downstream resources that require a different token audience, the server performs an OAuth 2.0 On-Behalf-Of (OBO) token exchange — trading the inbound user token for a token scoped to the target resource — and uses that exchanged token to execute the fetch.

Consumer OAuth Providers The server supports OAuth 2.1 with PKCE for consumer providers (Google, GitHub, and others). The user authenticates via the standard OAuth authorization code flow through their AI client. The resulting access token is stored in the user's session context and used by the server to execute fetches on behalf of the user for the duration of the session.

Cookie/Session Forwarding For sites where the user has an active browser session, the server accepts a serialized session state (cookies and localStorage) captured from the user's authenticated browser session. The server replays the session credentials when fetching the target URL. This mechanism is intended for local or single-user deployments where formal OAuth flows are not available for the target site.

Behavior

Authenticated fetch applies the same entitlement and classification checks as standard fetch. Authentication does not bypass access controls — a user with a valid session cannot fetch content from a blocked domain or a URL that fails classification policy.

Token handling follows the MCP Authorization Specification (2025-11-25). Tokens are validated on each request. Token passthrough to downstream services without validation is not permitted. The server maintains a consent registry mapping user identities to approved tool scopes.

Acceptance Criteria

- Given a valid enterprise IdP token in the tool call context, when an authenticated fetch is requested for a permitted URL, then the server executes the fetch using the user's identity and returns the content
- Given a target resource requiring a different token audience, when an authenticated fetch is requested, then the server performs an OBO token exchange before executing the fetch
- Given a valid consumer OAuth session, when an authenticated fetch is requested for a permitted URL, then the server executes the fetch using the user's OAuth credentials

- Given an authenticated fetch request for a URL on the deny list, when the fetch is requested, then a policy rejection is returned regardless of the user's authentication status
- Given an expired or invalid token, when an authenticated fetch is requested, then a structured authentication error is returned and no fetch is executed
- Given a user has not granted consent for the requesting client scope, when an authenticated fetch is requested, then the request is rejected with a consent-required response

---

Cross-Cutting Behaviors

Audit Logging

All tool calls — search queries, fetch requests, classification decisions, entitlement rejections, and authentication events — are logged. Log entries include: timestamp, tool called, caller identity (where available), query or URL, entitlement outcome, classification outcome, and response status. Logs are written to a configurable destination (stdout, file, external log sink).

Rate Limiting

The server enforces configurable rate limits per caller identity or per deployment. Rate limit thresholds are set by the Platform Administrator. When a rate limit is exceeded, a structured rate-limit response is returned. Rate limiting applies independently to search and fetch tool calls.

Transport

The server supports both stdio transport (for local MCP client integration) and HTTP/SSE transport (for remote and multi-client deployments). Transport mode is configured at deployment time.

Error Responses

All error conditions return a structured MCP tool response — never an unhandled exception. Every error response includes: error type, human-readable reason, and where applicable the specific policy, classification, or system condition that caused the failure.

03 — Server Surface

Product: MAOS Internet Access MCP Server
Version: 1.0
Date: 2026-06-16
Author: Andrew Bush / M&A Operating System

Protocol and Capability Declaration

Transport: stdio (local deployments) and Streamable HTTP with SSE (remote and multi-client deployments). Message format: JSON-RPC 2.0. Auth: OAuth 2.1 + PKCE with RFC 8707 resource binding for remote deployments. Conforms to MCP Authorization Specification 2025-11-25.

```
{
  "protocolVersion": "2025-03-26",
  "capabilities": {
    "tools": { "listChanged": false }
  },
  "serverInfo": {
    "name":    "maos-internet-mcp-server",
    "title":   "MAOS Internet Access MCP Server",
    "version": "1.0.0"
  },
  "instructions": "Provides controlled internet search and page fetch. All requests are subject to
entitlement and site classification policy. Use search to retrieve ranked web results, fetch to
retrieve page content in raw, markdown, chunked, or summarized format. Authenticated fetch
requires a valid identity token and is available from v1."
}
```

Tools

`search`

Search the web and return a ranked list of results filtered through the active entitlement and classification policy.

Input schema

Parameter	Type	Required	Description
`query`	string	Yes	The search query

Parameter	Type	Required	Description
`num_results`	integer	No	Number of results to return; default and maximum are deployment-configurable
`language`	string	No	Language preference for results
`time_range`	enum	No	Restrict results by recency: `day`, `week`, `month`, `year`, `any`

Response — `content`

Plain-text summary of result count and any applied filters.

Response — `structuredContent`

```
{
  "results": [
    {
      "title": "string",
      "url": "string",
      "snippet": "string",
      "source_domain": "string",
      "classification": {
        "category": "string",
        "reputation_score": "number",
        "safety_flags": ["string"],
        "source": "string"
      }
    }
  ],
  "filter_applied": "boolean",
  "total_before_filtering": "integer",
  "provenance": {
    "query": "string",
    "executed_at": "string",
    "backend": "string",
    "result_count_returned": "integer",
    "result_count_before_filtering": "integer",
    "results_hash": "string"
  }
}
```

`provenance.executed_at` is an ISO 8601 UTC timestamp. `provenance.results_hash` is a SHA-256 hex digest of the canonically serialized result set after filtering — enabling callers and audit systems to verify the result set has not been modified in transit. `provenance.backend` identifies the search backend that served the query (e.g. `searxng`, `brave`, `enterprise-index`).

Error conditions

Condition	Error type
Search backend unavailable	`backend_unavailable`

Condition	Error type
Query blocked by entitlement policy	`policy_rejection`
Zero results after filtering	Empty `results` array; `filter_applied: true`

`fetch`

Fetch a URL and return page content in the requested output format, subject to entitlement and classification checks.

Input schema

Parameter	Type	Required	Description
`url`	string	Yes	The URL to fetch
`format`	enum	No	Output format: `raw`, `markdown`, `chunked`, `summarized`. Default: `markdown`
`chunk_size`	integer	No	Token or character limit per chunk; only used when `format` is `chunked`
`start_chunk`	integer	No	Chunk index to begin from; enables pagination through large pages

Output formats

Format	Description
`raw`	Unprocessed HTTP response body; all HTML, scripts, and structure preserved
`markdown`	Page content converted to clean Markdown; navigation, advertisements, and boilerplate stripped
`chunked`	Markdown content segmented into discrete chunks; each chunk includes its index and total chunk count
`summarized`	LLM-generated summary of the page content; requires a configured LLM backend

Response — `content`

The fetched page content in the requested format.

Response — `structuredContent`

```
{
  "format": "string",
  "classification": {
    "category": "string",
    "reputation_score": "number",
    "safety_flags": ["string"],
    "source": "string"
  },
  "chunk_index": "integer | null",
  "chunk_total": "integer | null",
  "provenance": {
    "url": "string",
    "fetched_at": "string",
    "content_length_bytes": "integer",
    "content_hash": "string",
    "http_status": "integer",
    "redirected_from": "string | null"
  }
}
```

`provenance.fetched_at` is an ISO 8601 UTC timestamp. `provenance.content_hash` is a SHA-256 hex digest of the raw response body before any format conversion — enabling callers and audit systems to verify the content returned matches what the server received from the origin. `provenance.redirected_from` records the original URL when the target server issued a redirect.

Error conditions

Condition	Error type
URL blocked by entitlement policy	`policy_rejection`
URL blocked by site classification	`classification_rejection`
Target server non-200 response	`fetch_error` with HTTP status code
Target server timeout	`timeout_error`
`summarized` format with no LLM backend configured	`capability_unavailable`
`start_chunk` beyond available chunks	`chunk_out_of_range`

`fetch_authenticated`

Fetch a URL using the caller's identity credentials. Available from v1. Applies the same entitlement and classification checks as `fetch` — authentication does not bypass access controls.

Input schema

Parameter	Type	Required	Description
`url`	string	Yes	The URL to fetch
`format`	enum	No	Output format: `raw`, `markdown`, `chunked`, `summarized`. Default: `markdown`
`chunk_size`	integer	No	Token or character limit per chunk
`start_chunk`	integer	No	Chunk index to begin from
`token`	string	No	Bearer token from the enterprise IdP; used for OBO token exchange if the target resource requires a different audience

Authentication modes

Mode	Mechanism
Enterprise IdP	Bearer token accepted from any OIDC-compliant provider; OBO exchange performed for resources requiring a different token audience
Consumer OAuth (v2)	OAuth 2.1 with PKCE; access token from the authorization code flow stored in session context
Cookie/session forwarding (v2)	Serialized session state (cookies, localStorage) replayed against the target URL

Response shapes

Identical to `fetch`. An additional `auth_method` field is included in `structuredContent` indicating which authentication mode was used. The `provenance` block is always present and identical in structure to `fetch` — authentication does not alter provenance recording.

Error conditions

Condition	Error type
Invalid or expired token	`auth_error`
Consent not granted for requesting client scope	`consent_required`
URL blocked by entitlement or classification policy	`policy_rejection` / `classification_rejection`

Cross-Cutting Behaviors

Error Response Shape

All error conditions return a structured MCP tool response. No unhandled exceptions are surfaced to the caller.

```
{
  "error_type": "string",
  "reason": "string",
  "detail": {}
}
```

Provenance

Every successful `search`, `fetch`, and `fetch_authenticated` response includes a `provenance` block in `structuredContent`. The provenance block is the authoritative record of what was requested, when, and what was returned:

Field	Present in	Description
`query`	`search`	The exact query string submitted to the backend
`executed_at`	`search`	ISO 8601 UTC timestamp of query execution
`backend`	`search`	Identifier of the search backend that served the query
`result_count_returned`	`search`	Number of results returned after filtering
`result_count_before_filtering`	`search`	Number of results received from the backend before entitlement and classification filtering
`results_hash`	`search`	SHA-256 hex digest of the canonically serialized filtered result set
`url`	`fetch`, `fetch_authenticated`	Final URL fetched, after any server-side redirects
`fetched_at`	`fetch`, `fetch_authenticated`	ISO 8601 UTC timestamp of the fetch
`content_length_bytes`	`fetch`, `fetch_authenticated`	Size of the raw response body in bytes
`content_hash`	`fetch`, `fetch_authenticated`	SHA-256 hex digest of the raw response body before format conversion

Field	Present in	Description
`http_status`	`fetch`, `fetch_authenticated`	HTTP status code returned by the origin server
`redirected_from`	`fetch`, `fetch_authenticated`	Original URL if a redirect occurred; `null` otherwise

The provenance block is returned to the caller in every successful response and is independently recorded in the audit log. This enables downstream consumers — agents, audit systems, and compliance tooling — to verify that the content they received matches what the server retrieved from the origin, and to establish an unambiguous chain of custody from request to response.

Audit Logging

Every tool call — including policy rejections, classification blocks, and authentication failures — is written to the audit log. Each audit log entry includes: timestamp, tool name, caller identity (where available), query or URL, entitlement outcome, classification outcome, response status, and the full provenance block for successful responses. Log destination is configurable at deployment time (stdout, file, or external log sink).

Rate Limiting

Configurable per caller identity and per tool. A structured `rate_limit_exceeded` response is returned when the threshold is reached.

04 — Technical Constraints and Deployment

Product: MAOS Internet Access MCP Server
Version: 1.0
Date: 2026-06-16
Author: Andrew Bush / M&A Operating System

Deployment Context

The server is deployed as self-hosted infrastructure within an enterprise environment. It has no dependency on any external managed service for its core search and fetch capabilities. All components required for operation run within the enterprise's own infrastructure boundary.

The server is designed to run within existing enterprise network and security controls. It does not bypass or replace those controls — it operates within them and adds AI-specific access governance on top.

MCP Protocol Conformance

The server implements the Model Context Protocol specification and exposes capabilities exclusively as MCP tools. It conforms to the MCP Authorization Specification (2025-11-25) for all authenticated interactions.

Requirement	Implementation
MCP tool interface	All capabilities exposed as standard MCP tools
Transport: stdio	Supported — for local and single-client deployments
Transport: HTTP/SSE	Supported — for remote and multi-client deployments
Transport: Streamable HTTP	Supported — MCP spec 2025-03-26 and later
OAuth 2.1 with PKCE	Required for authenticated fetch via consumer providers
OAuth 2.0 OBO flow	Required for authenticated fetch via enterprise IdP
Dynamic client registration	Supported per MCP Authorization Specification
Protected resource metadata	Exposed at `/.well-known/oauth-protected-resource`

Search Backend

The search backend is pluggable. The server abstracts the backend behind a standard internal interface so that the MCP tool surface is identical regardless of which backend is active. The backend is selected and configured at deployment time by the Platform Administrator.

Target state supported backends:

Backend Type	Description
Self-hosted metasearch (e.g. SearXNG)	Aggregates results from multiple search engines; no external API key required; fully self-hosted
Search API (e.g. Brave Search, Tavily)	External search API; API key required; managed by the vendor
Enterprise search index	Internal enterprise search system; for deployments requiring search over internal and external content

The server does not require any specific backend. Deployments that require zero external API dependency configure a self-hosted metasearch backend.

Fetch Backend

Static page fetch is performed by the server's own HTTP client. No browser automation is required for static HTML content.

For JavaScript-rendered pages (single-page applications, dynamic content), a browser automation backend is required. The browser automation backend is pluggable and configured at deployment time.

Content Type	Fetch Mechanism
Static HTML	Built-in HTTP client
JavaScript-rendered pages	Configured browser automation backend (e.g. Playwright, Puppeteer)
PDFs and documents	Built-in document parser

Site Classification Backend

The classification backend is pluggable. One or more classification providers are configured at deployment time. Where multiple providers are configured, the most restrictive classification result applies.

Target state supported classification sources:

Source Type	Description
Commercial classification database (e.g. zvelo, BrightCloud/OpenText)	High-coverage URL and domain classification with reputation scoring; OEM/partner access required
Open threat feeds (e.g. Google Safe Browsing, URLhaus, PhishTank, Spamhaus)	Free or low-cost feeds covering malware, phishing, and spam; primarily security-focused
Enterprise classification policy	Custom category and domain rules defined by the enterprise and applied locally

Summarisation Backend

The summarized fetch output format requires a configured language model backend. The LLM backend is pluggable and configured at deployment time. Deployments that do not configure an LLM backend return an error for summarized fetch requests.

Compliance and Security

Area	Requirement
Token handling	Tokens validated on every request per OAuth 2.1; no token passthrough to downstream services
Consent	User consent recorded per client and scope; consent registry maintained server-side
Audit logging	All tool calls, entitlement decisions, and classification decisions logged
Data in transit	All HTTP communication over TLS
Credentials	No credentials stored in plaintext; environment-variable or secrets-manager injection at deploy time
GDPR	Search queries and fetch URLs constitute user activity data; log retention policy configurable
Enterprise perimeter	Server operates within and subject to enterprise network and identity controls

Operational Requirements

Requirement	Detail
Deployment packaging	Docker container; environment-variable-driven configuration

Requirement	Detail
Configuration surface	All configurable parameters set via environment variables or mounted configuration file; no runtime UI
Health endpoint	Exposed health and readiness endpoints for integration with enterprise monitoring
Rate limiting	Configurable per caller identity and per tool; enforced server-side
Timeout configuration	Per-tool timeouts configurable; defaults provided
Log destination	Configurable: stdout, file, or external log sink (e.g. syslog, SIEM integration)

Locale

The server itself has no locale-specific behavior. Search result language preferences are passed through to the backend where supported. Fetch content is returned in the language of the source page. No translation capability is provided by the server.

Accessibility

Not applicable. The server exposes no user-facing interface. All interaction is via the MCP tool protocol.

05 — Roadmap

Product: MAOS Internet Access MCP Server
Version: 1.0
Date: 2026-06-16
Author: Andrew Bush / M&A Operating System

MVP — Initial Release

The MVP establishes the core capability pair — search and fetch — with the content safety controls necessary for enterprise deployment. The MVP is deployable as production infrastructure immediately upon release.

Search - Query submission and ranked result return - Configurable result count and time range filtering - Pluggable search backend (self-hosted metasearch or search API) - Entitlement filtering applied to results before return

Fetch - URL fetch with output format selection: raw and markdown - Boilerplate stripping and HTML-to-Markdown conversion - Configurable timeout and error handling - Entitlement checks before fetch execution

Entitlements - Domain allow list and deny list - URL pattern rules - Configurable default posture (open or closed) - Policy rejection responses with reason

Site Classification - Integration with at least one open threat feed (Google Safe Browsing, URLhaus, or equivalent) - Safety flag enforcement: malware, phishing blocked by default - Classification metadata returned alongside results and fetch responses - Configurable classification threshold policy

Operational Foundations - stdio and HTTP/SSE transport - Audit logging to stdout and file - Docker packaging with environment-variable configuration - Health and readiness endpoints - Rate limiting per caller

---

v1 — Capability Completion

v1 completes the fetch output format set, extends classification coverage, and adds authenticated fetch for enterprise identity providers.

Fetch extensions - Chunked output format with pagination support - Summarized output format (requires configured LLM backend) - Browser automation backend for JavaScript-rendered pages - PDF and document fetch and parsing

Classification extensions - Integration with commercial classification database (zvelo, BrightCloud/ OpenText, or equivalent) - Multi-provider classification with most-restrictive-wins logic - Content category

blocking (adult, gambling, hate speech, and configurable categories) - Reputation score threshold enforcement

Authenticated Fetch — Enterprise IdP - OAuth 2.1 / OIDC token acceptance from enterprise identity providers (Entra ID, Okta) - On-Behalf-Of (OBO) token exchange for downstream resource access - Consent registry per user and client scope - MCP Authorization Specification (2025-11-25) conformance

Operational extensions - External log sink integration (syslog, SIEM) - Per-tool rate limit configuration - Configurable log retention policy

---

v2 — Authenticated Fetch Completion and Ecosystem Integration

v2 extends authenticated fetch to consumer OAuth providers and adds integration surface for enterprise search and classification ecosystems.

Authenticated Fetch — Consumer OAuth - OAuth 2.1 with PKCE for consumer providers: Google, GitHub, and configurable providers - Cookie/session forwarding for sites without OAuth support - Dynamic client registration for MCP-compatible clients

Enterprise search backend - Integration with enterprise internal search indexes - Unified result set combining internal and external search results

Classification ecosystem - Pluggable classification provider API for custom enterprise classification sources - Classification override rules for enterprise-specific domain policies

Observability - Structured telemetry output (OpenTelemetry) - Per-tool usage metrics - Classification decision audit trail with provider attribution

---

Future Considerations

Capabilities identified as potentially valuable but not yet scheduled:

- Caching layer for frequently accessed pages with configurable TTL
- Search result re-ranking using a local ML model
- Automatic language detection and translation of fetched content
- Webhook or event notification for classification policy violations
- Multi-tenant deployment model with per-tenant entitlement and classification policy isolation

06 — Open Decisions

Product: MAOS Internet Access MCP Server
Version: 1.0
Date: 2026-06-16
Author: Andrew Bush / M&A Operating System

Decisions that must be resolved before or during build. Each decision has a priority indicating the urgency of resolution relative to the build sequence.

#	Decision Needed	What It Affects	Priority
1	Which self-hosted metasearch engine is the reference implementation for MVP search backend? The research identified SearXNG as the strongest candidate but this should be confirmed as the default before build begins.	MVP search capability; deployment documentation; Docker compose configuration	HIGH
2	Which open threat feed(s) are bundled as the default classification provider in MVP? Google Safe Browsing, URLhaus, and PhishTank are candidates. The selection affects what is blocked by default and the completeness of MVP safety coverage.	MVP site classification; default safety posture; deployment defaults	HIGH
3	What is the default fetch posture for JavaScript-rendered pages in MVP — fail with an informative error, or partially return available static content? This affects whether a browser automation backend is a hard MVP dependency.	MVP fetch capability; deployment complexity; user experience for JS-heavy sites	HIGH
4	What is the default entitlement posture — open (deny list only) or closed (allow list required)? This is a security architecture decision that affects how the server behaves on first deployment before an administrator has configured rules.	Entitlements; first-run behavior; security posture	HIGH
5	What LLM backend is supported for summarized fetch in v1? The summarization capability requires a configured language model — the server needs to define which providers are supported (local model, OpenAI-compatible API, etc.) and what the default is.	v1 fetch — summarized format; deployment requirements	MEDIUM
6	How is the classification threshold policy expressed in configuration? A decision is needed on whether thresholds are defined as numeric score cutoffs, category blocklists, named policy profiles, or a combination. This affects the configuration schema and the administrator experience.	Entitlements; site classification; operator UX	MEDIUM

#	Decision Needed	What It Affects	Priority
7	What is the audit log schema? The log format needs to be defined before build so that downstream SIEM and observability integrations can be designed against a stable schema.	Audit logging; operational integration; compliance	MEDIUM
8	Which enterprise IdP providers are explicitly tested and documented for v1 authenticated fetch? Entra ID and Okta are the expected primary targets but the scope of tested providers should be confirmed before v1 build.	v1 authenticated fetch; enterprise integration documentation	MEDIUM
9	What consumer OAuth providers are supported in v2 authenticated fetch beyond Google and GitHub? The scope of supported providers affects the OAuth proxy and dynamic client registration implementation.	v2 authenticated fetch	LOW
10	Is multi-tenant entitlement and classification policy isolation a v2 or future consideration? If multiple business units within an enterprise need isolated policies, this significantly affects the configuration and identity model.	Roadmap; future architecture	LOW